

LangGraph & RAG — Building a Multi-Step Pipeline

Week 3 — Turing x UCSB x ACM

Irene Fan & Agnibha Sarkar | March 13, 2026

irenefan@turing.com · agnibha.sarkar@turing.com

Schedule

Week	Topic
1	Course Kickoff + Problem Scoping
2	Agentic AI & Fundamentals
3	LangGraph & RAG
4	HITL Systems
5	Industry Realities of Building AI Products
6	Architecture Design & Eng Design Doc
7	Architecture Presentations + Critique
8	Coding Copilots & Dev Cycle Integration
9	End-to-End Implementation & Evaluation
10	Demo Week

Today

1. Recap Week 2 — Quiz & Discussion Revisited
 2. From Extraction to Classification
 3. RAG: Why, Not Just How
 4. LangGraph: Graphs Over Glue Code
 5. Live Demo: CBRE Extract → Retrieve → Classify
 6. Discussion
 7. Assignment + Reading List
-

Quick Recap: Week 2

Last week we covered:

- **Agentic AI** — iterative workflows, not one-shot prompts
- **Four design patterns** — reflection, tool use, planning, multi-agent
- **Reflection deep dive** — review loops that catch compound hazards

- **Evaluations** — measure systematically, the #1 predictor of success
- **Hands-on** — entity extraction from maintenance call transcripts

We built the **extraction node**. Today we connect it to the rest of the pipeline.

Let's Revisit: Food for Thought from Week 2

FT-2 (from last week)

Reflection improves results, but every reflection step is another LLM call — more cost, more latency.
When is reflection *not* worth it?

Did anyone think about this during the week?

FT-5 (from last week)

We used one LLM to generate and the same LLM to reflect. **Is the model really "catching its own mistakes," or is it just getting a second chance with more context?**

Does it matter if the result improves either way?

From Extraction to Classification

Last week: **Transcript → Entities** (problem type, severity, location)

But extraction alone doesn't route the call. We need to:

1. **Match** the extracted problem to a known problem code
2. **Classify** using both the extraction and the matched codes

Our pipeline grows:

```
Transcript → Extract Entities → Retrieve Problem Codes → Classify → Route
```

The question is: **how do we find the right problem codes?**

The Naive Approach: Stuff Everything in the Prompt

We have 21 problem codes. Why not just list them all in the prompt?

```
"Here are all 21 problem codes. Based on this transcript, pick the best one."
```

This works. For 21 codes.

But What If...

- CBRE has **500** problem codes across different regions?
- Each code has a **paragraph-long description** + keywords?
- You want to add **historical work orders** as context?
- The prompt hits **100,000+ tokens** — every single call?

The prompt becomes a haystack. The LLM has to find the needle.

RAG: Retrieval-Augmented Generation

The idea is simple:

1. **Store** your knowledge in a searchable format
2. **Retrieve** only the relevant pieces for each query
3. **Generate** a response using only the retrieved context

Instead of stuffing everything in the prompt, you search first, then pass just what's relevant.

Why RAG Matters

Without RAG	With RAG
All 500 codes in every prompt	Only 3-5 relevant codes per call
Cost scales with total knowledge	Cost scales with relevance
LLM attention diluted across everything	LLM focuses on what matters
Adding more codes = bigger prompts	Adding more codes = bigger index (cheap)
Works for 21 codes	Works for 500,000 codes

How RAG Works — The Key Insight

RAG relies on **embeddings** — turning text into numbers that capture meaning.

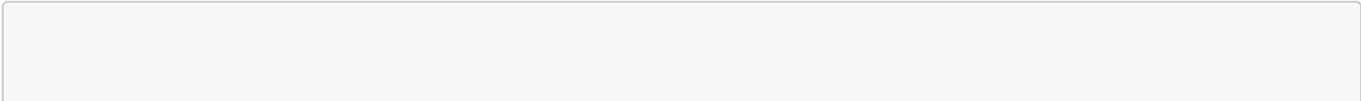
"**Water leaking from ceiling pipe burst**" → vector: [0.23, -0.14, 0.87, ...]

"**PLUMB-001: Pipe Leak / Burst Pipe — Water leaking from pipes...**" → vector: [0.21, -0.12, 0.85, ...]

These vectors are **close** in embedding space because the meaning is similar.

A search query doesn't match keywords — it matches **meaning**.

The RAG Pipeline



1. **LOAD** — Read your documents (problem codes, manuals, etc.)
 2. **SPLIT** — Break large docs into smaller chunks
 3. **EMBED** — Convert each chunk to a vector
 4. **STORE** — Save vectors in a vector store (ChromaDB, Pinecone, etc.)
 5. **RETRIEVE** — For each query, find the k nearest vectors
 6. **GENERATE** — Feed the retrieved chunks + question to the LLM

Steps 1-4 happen **once** (indexing). Steps 5-6 happen **every query**.

Is RAG Still Needed?

Context windows keep growing — Claude handles ~200K tokens, Gemini supports 1M+.

If models can read everything directly, why bother with retrieval?

The Case for Long Context

Long context is a "no stack stack" — no embedding models, no vector databases, no chunking strategy.

It also solves two problems RAG struggles with:

- **Retrieval lottery** — RAG can silently fail when retrieval misses the right chunk, even if it's in the database. Long context avoids this by including everything.
- **The "whole book" problem** — RAG is bad at identifying *gaps* or comparing across an entire document. Long context lets the model see the full picture.

The Case for RAG

Three reasons RAG isn't going away:

1. **Compute efficiency** — Long context re-reads the *entire* document for every query. RAG processes data once during indexing, then retrieves only what's needed.
2. **Attention dilution** — As context grows, the model's attention gets spread thin. Research shows accuracy drops on specific facts buried in massive contexts ("needle in a haystack"). RAG removes the noise and shows only the signal.
3. **Infinite datasets** — A million tokens sounds like a lot, but enterprise data is measured in terabytes. A retrieval layer is the only way to navigate data at that scale.

When to Choose What

Choose Long Context When...	Choose RAG When...
Bounded dataset (one contract, one book)	Vast knowledge base (500+ docs, enterprise data)
Complex global reasoning across full doc	Repeated queries over the same data
Simplicity matters more than cost	Cost and latency are priorities

Choose Long Context When...	Choose RAG When...
One-off analysis tasks	Production system at scale

For CBRE with 500 codes + work orders + manuals: **RAG**.

Watch: [RAG vs. Long Context — IBM Technology](#) Read: [Is RAG Still Needed? — Alex Wang](#)

Now: How Do We Wire This Together?

We have:

1. An extraction function (Week 2)
2. A retriever (RAG — today)
3. A classifier (today)

We need to connect them into a **pipeline** where data flows from one step to the next.

We *could* just call functions in sequence. But...

Why Not Just Chain Functions?

```
entities = extract(transcript)
codes = retrieve(entities)
classification = classify(entities, codes)
```

This works for a linear pipeline. But what happens when:

- Retrieval returns irrelevant results and you need to **retry with a different query**?
- The classifier's confidence is low and you want to **route to a human**?
- You want to **add a reflection step** between extraction and retrieval?
- Different transcript types need **different paths** through the pipeline?

You need **conditional routing**. And that's where simple function chaining breaks.

LangGraph: Graphs Over Glue Code

LangGraph models your pipeline as a **graph**:

- **Nodes** — Python functions that do one thing (extract, retrieve, classify)
 - **Edges** — Connections between nodes (extract → retrieve)
 - **State** — A shared dictionary that flows through the graph
 - **Conditional edges** — "If confidence < 0.7, go to human review instead of auto-route"
-

The Mental Model

Think of it like a flowchart:

```
START
↓
[Extract Entities]
↓
[Retrieve Problem Codes] ←—— retry if irrelevant
↓
[Classify Problem]
↓
├── confidence ≥ 0.7 → [Auto-Route] → END
└── confidence < 0.7 → [Human Review] → END
```

Each box is a **node**. Each arrow is an **edge**. The branch is a **conditional edge**.

LangGraph makes this explicit in code — not buried in if/else spaghetti.

State: The Pipeline's Memory

```
class PipelineState(TypedDict):
    transcript: str
    entities: Optional[dict]
    retrieved_codes: Optional[list[dict]]
    classification: Optional[dict]
```

Every node reads from state and writes back to it.

- `extract_entities` reads `transcript`, writes `entities`
- `retrieve_codes` reads `entities`, writes `retrieved_codes`
- `classify_problem` reads `entities` + `retrieved_codes`, writes `classification`

No global variables. No passing 10 arguments between functions. Just state.

Nodes: Just Python Functions

```
def extract_entities(state: PipelineState) -> dict:
    result = llm.with_structured_output(MaintenanceEntity).invoke(...)
    return {"entities": result.model_dump()}

def retrieve_codes(state: PipelineState) -> dict:
    docs = retriever.invoke(query)
    return {"retrieved_codes": codes}

def classify_problem(state: PipelineState) -> dict:
    result = llm.with_structured_output(Classification).invoke(...)
    return {"classification": result.model_dump()}
```

Each node returns **only the keys it updates**. LangGraph merges them into state.

Wiring the Graph

```
workflow = StateGraph(PipelineState)

workflow.add_node("extract", extract_entities)
workflow.add_node("retrieve", retrieve_codes)
workflow.add_node("classify", classify_problem)

workflow.add_edge(START, "extract")
workflow.add_edge("extract", "retrieve")
workflow.add_edge("retrieve", "classify")
workflow.add_edge("classify", END)

pipeline = workflow.compile()
```

Today's demo: a linear pipeline.

Week 4: we add **conditional edges** for human-in-the-loop.

Why This Matters

Just Functions	LangGraph
Implicit flow control	Explicit, visual graph
Hard to add branching	Conditional edges are first-class
State passed manually	Shared state object
Hard to test individual steps	Each node testable in isolation
Hard to visualize	<code>graph.get_graph().draw_mermaid_png()</code>
Can't easily add retry/loop logic	Built-in support for cycles

Demo Time

`langgraph_rag_demo.ipynb`

```
Transcript → Extract Entities → RAG Retrieve Codes → Classify → Evaluate
```

We'll walk through each node, run a single transcript, then evaluate across all 10.

Quiz: Check Your Understanding

Q1

CBRE currently has 21 problem codes, and your prototype stuffs all 21 into the prompt (~1,200 tokens of context). It works well. A product manager says: "Why add the complexity of a vector store?" Which is the **strongest** argument for switching to RAG now?

- (A) Vector stores are always faster than in-context lookup
 - (B) The system will need to scale — when CBRE adds regional codes, historical work orders, and equipment manuals, the prompt will exceed context limits. Building the retrieval layer now prevents a costly rewrite later
 - (C) RAG is more accurate than in-context lookup for any number of documents
 - (D) The vector store eliminates hallucination entirely
-

Q2

You embed your 21 CBRE problem codes and search for "the office smells weird, kind of musty." The retriever returns: (1) HVAC-003: Air Quality/Ventilation, (2) SAFE-001: Gas Leak/Chemical Hazard, (3) JANI-001: Restroom Supplies/Cleaning. Which result is most relevant, and what does this illustrate?

- (A) SAFE-001 — "smell" is closest to "gas" in embedding space, proving embeddings are always correct
 - (B) HVAC-003 — "musty" semantically relates to air quality issues, showing embeddings capture meaning beyond keywords. But SAFE-001's presence shows embeddings can't distinguish between "musty smell" (mold) and "dangerous smell" (gas) without more context
 - (C) JANI-001 — cleaning issues are the most common cause of office smells, and frequency matters most in retrieval
 - (D) All three are equally relevant — the classifier downstream will sort it out, so retrieval quality doesn't matter
-

Q3

Your LangGraph pipeline has three nodes: extract → retrieve → classify. The classifier produces a wrong answer because the retriever returned irrelevant codes. With plain function chaining, you'd add an if-statement. In LangGraph, what's the right approach?

- (A) Add a try/except in the classify node to catch bad inputs
 - (B) Add a conditional edge after the retrieve node that grades document relevance — if irrelevant, route to a "rewrite query" node that loops back to retrieve; if relevant, proceed to classify
 - (C) Increase k in the retriever to return more codes — more results always means better results
 - (D) Add a reflection step inside the retrieve node itself
-

Q4

In `PipelineState`, you define `retrieved_codes: Optional[list[dict]]`. The retrieve node returns `{"retrieved_codes": codes}`. A student asks: "Why not just return the codes directly and pass them as an argument to classify?" What's the key advantage of using LangGraph's state?

- (A) State is faster than function arguments because it uses shared memory
 - (B) State makes the data flow explicit and persistent — any node can read any previous node's output without changing function signatures, and you can inspect the full state at any point for debugging. Function arguments create brittle chains where adding a new node means changing every downstream signature
 - (C) State is required by Python — you can't pass dictionaries between functions
 - (D) State automatically validates types, preventing all runtime errors
-

Q5

You're tuning your RAG system. With `chunk_size=100` tokens and `k=3`, retrieved fragments are too small to be meaningful — the classifier often misclassifies. With `chunk_size=2000` and `k=3`, chunks contain multiple problem codes mixed together, diluting relevance. What's the right approach?

- (A) Always use the smallest chunk size possible for maximum granularity
 - (B) Set `chunk_size` to match the natural unit of your data — each CBRE problem code is a self-contained document (~50-100 tokens), so one chunk = one code. The "right" chunk size depends on data structure, not a universal rule
 - (C) Use the largest chunk size your context window allows
 - (D) Chunk size doesn't matter if you increase `k` enough
-

Q6

Your pipeline runs on TX-009 (roof leak dripping into server room). The retriever correctly returns PLUMB-002 (Roof Leak) as the top result. But the classifier assigns PLUMB-001 (Pipe Leak) because the transcript mentions "dripping" and "water damage." What would help?

- (A) Remove PLUMB-001 from the vector store so it can never be retrieved
 - (B) Increase the retriever's `k` value — more options always helps
 - (C) The classifier prompt needs to emphasize distinguishing between retrieved codes based on the root cause in the transcript, not surface-level keyword overlap. The retriever did its job; the generation step needs better instructions
 - (D) Switch from embeddings to keyword matching — it would catch "roof" directly
-

Q7

You add a conditional edge after classification: if `confidence < 0.7`, route to "human_review" instead of "auto_route." This reveals a broader principle about LangGraph. Which statement best captures it?

- (A) Conditional edges are just if-statements — LangGraph adds no value over plain Python
- (B) Conditional edges make decision logic part of the graph's structure, not hidden inside node code. The workflow is inspectable and modifiable — you can see all possible paths and change routing without touching the nodes

- (C) Conditional edges only work with numeric thresholds, not LLM-based decisions
 - (D) Conditional edges always require a human to make the decision
-

Q8

In the LangChain agentic RAG tutorial, the agent retrieves documents, then grades them. If irrelevant, it rewrites the question and tries again. Which agentic design pattern from Week 2 does the "grade → rewrite → retry" loop most closely resemble?

- (A) Multi-agent collaboration — the grader and rewriter are separate agents
 - (B) Planning — the system decides what steps to take based on intermediate results
 - (C) Reflection — the system evaluates its own output (the retrieval result), identifies a problem (irrelevant documents), and corrects course (rewrites the query). It's a feedback loop applied to retrieval rather than generation
 - (D) Tool use — the retriever is a tool being called
-

Q9

Your system indexes 500 problem codes. A transcript describes "loud banging from the pipes whenever someone flushes on the 3rd floor." The top 3 retrieved codes are PLUMB-001 (Pipe Leak), PLUMB-003 (Toilet/Restroom), and HVAC-003 (Air Quality). None perfectly match "water hammer" which isn't in the database. What does this reveal?

- (A) RAG fails whenever the exact answer isn't in the database — switch to a pure LLM approach
 - (B) RAG retrieves the most semantically similar items in its index, but can only return what's been indexed. Knowledge base gaps mean plausible-but-wrong matches. This is why RAG systems need monitoring, knowledge base maintenance, and a fallback for low-confidence retrievals
 - (C) The embedding model is broken — "pipe banging" should obviously match a plumbing code
 - (D) Increasing k to 10 would solve this
-

Q10

You run extract → retrieve → classify on all 10 transcripts: 70% accuracy. You build the agentic RAG version (document grading + query rewriting): accuracy jumps to 85%. A teammate says: "The improvement is just because the agentic version makes more LLM calls." What's actually driving it?

- (A) They're right — more LLM calls always means better results
 - (B) The improvement comes from the system's ability to self-correct — when retrieval returns irrelevant codes, the system detects it (grading), reformulates (rewriting), and retries. It's not more calls — it's smarter calls
 - (C) The agentic version uses a bigger model, not more calls
 - (D) The improvement is random variance — run it again and the numbers will equalize
-

Quiz Answers

Q	Answer	Why
1	B	21 codes work in-context today, but the system must scale. Building retrieval now avoids a rewrite when the knowledge base grows 10x–100x. The strongest argument is forward-looking, not about current performance
2	B	Embeddings capture semantic similarity ("musty" ↔ "air quality") which keyword matching would miss. But they also surface SAFE-001 because "smell" is semantically close to "gas leak" — similarity ≠ relevance. The classifier needs context to disambiguate
3	B	Conditional edges are LangGraph's answer to "what if a step produces bad results." Grade documents, route accordingly. This is the same pattern as the agentic RAG tutorial's grade → rewrite → retry loop
4	B	State decouples nodes from each other. Adding a new node (say, "reflection") doesn't require changing function signatures upstream or downstream. It also enables debugging — inspect state at any point
5	B	Chunk size should match data structure. CBRE problem codes are natural documents — one code per chunk. For long documents (manuals), you'd split differently. There is no universal optimal chunk size
6	C	Retrieval was correct (PLUMB-002 was returned). The problem is in generation — the classifier needs a better prompt that focuses on root cause, not keyword overlap. Fixing the wrong step is a common mistake
7	B	Conditional edges externalize decision logic into the graph structure, making workflows visible and modifiable. In plain Python, routing logic gets buried in nested if/else chains inside functions
8	C	The grade → rewrite → retry loop is reflection applied to retrieval. The system evaluates its own intermediate output, detects a quality issue, and corrects before proceeding — the same pattern as reviewing an extraction for severity errors
9	B	RAG can only return what's indexed. If the knowledge base has gaps, the system will return the closest match, which may be wrong. This is why production RAG systems need knowledge base coverage monitoring and low-confidence fallbacks
10	B	The key insight: more LLM calls without feedback loops don't help. The agentic version improves because it detects and corrects failures (bad retrieval → rewrite → retry). The calls are directed by evaluation, not random

Food for Thought

No right or wrong answers — open discussion prompts.

FT-1

We used RAG to search 21 problem codes. But context windows now support 200K+ tokens — you could fit all 21 codes (and their descriptions) in the prompt with room to spare. **At what scale does RAG become genuinely necessary vs. just a nice-to-have?** Is there a number of documents where the switch becomes obvious, or is it more nuanced than that?

FT-2

The agentic RAG tutorial adds a "grade documents" step that checks if retrieved results are relevant. This adds cost and latency to every query. **Should every RAG system include a retrieval quality check, or is it overkill for some use cases?** When would you skip it?

FT-3

Our pipeline is linear today: extract → retrieve → classify. But real maintenance calls are messy — multiple problems, ambiguous descriptions, missing info. **What would a non-linear version look like?** Where would you add branches, loops, or parallel paths?

FT-4

Embeddings capture semantic similarity — "water leak" is close to "pipe burst." But similarity isn't the same as relevance. **Can you think of a case where two things are semantically similar but retrieving one to answer a question about the other would be misleading?** What does this mean for how we evaluate RAG systems?

FT-5

We're building a system that routes maintenance calls automatically. But the first version of this system at CBRE was already automated — and it was calling 911 for leaky faucets. **How is what we're building fundamentally different from the old broken system?** Or is it just a fancier version of the same idea?

Assignment

Time estimate: ~3-4 hours (including setup, reading, and building)

Part 1: Build Your Own Agentic RAG System

Follow the official LangChain tutorial and adapt it to the CBRE problem:

Tutorial: [Build a Custom RAG Agent with LangGraph](#)

The tutorial builds a RAG agent that:

1. Decides whether to retrieve context or respond directly
2. Retrieves from a vector store
3. **Grades** whether the retrieved documents are relevant

- 4. **Rewrites** the query if they aren't
- 5. Generates a final answer

Your task: Adapt this to the CBRE domain.

What to Adapt

Tutorial Uses	You Should Use
Blog posts from Lilian Weng	CBRE <code>problem_codes.json</code> (21 codes)
<code>WebBaseLoader</code> for fetching docs	<code>json.load()</code> for problem codes
Blog-specific retriever tool	Problem code retriever tool
Generic question answering	Maintenance call classification

The data files are in the `data/` folder at the project root.

Milestones

Aim to get these working, in order:

- 1. **Index** the problem codes into a vector store and test retrieval
- 2. **Build `generate_query_or_respond`** — the LLM decides whether to retrieve
- 3. **Build `grade_documents`** — checks if retrieved codes are relevant
- 4. **Build `rewrite_question`** — improves the query if retrieval was poor
- 5. **Build `generate_answer`** — classifies using the relevant codes
- 6. **Wire the full graph** and test on 2-3 transcripts
- 7. **Stretch:** Run on all 10 transcripts and compare accuracy to last week's extraction-only approach

Part 2: Think About It

After building (or attempting) the system, answer these in a few sentences each:

- 1. When the LLM decides **not** to retrieve — was that the right call? Give an example.
- 2. Did the document grading step ever reject a relevant code or keep an irrelevant one?
- 3. Compare your accuracy this week (extraction + RAG + classification) to last week (extraction only). What changed?

What to Bring to Week 4

- Your working notebook (or however far you got + what you got stuck on)
- Your answers to the "Think About It" questions
- Ideas for where a human should review the pipeline's decisions (feeds into Week 4: HITL)

Reading List

Resource	What & Why
RAG vs. Long Context — IBM Technology	11-min video breaking down when to use RAG vs. long context windows. Great visual explainer.
Is RAG Still Needed? — Alex Wang	The RAG vs. long-context debate in article form. Helps you reason about <i>when</i> RAG is the right architecture.
Build a Custom RAG Agent (LangGraph)	Your assignment reference — the step-by-step tutorial to adapt.
LangGraph Documentation	Reference — nodes, edges, state, conditional routing.
LangChain RAG Conceptual Guide	Deeper dive into RAG architecture — embeddings, splitters, retrievers.
LangChain Docs Index	Master index for discovering all available LangChain API pages.
"We've raised \$6.5M to kill vector databases" — HydraDB	Provocative take on why vector search breaks at scale (10M+ docs) — similarity $\neq$ relevance. Introduces ontology-first context graphs as an alternative.
Gemini Embedding 2 — Google	Google's first natively multimodal embedding model — maps text, images, video, and audio into a single embedding space. Shows where embeddings are heading.
Can LLMs Critique and Iterate on Their Own Outputs? — Eric Jang	Deep exploration of LLM self-reflection with hands-on examples. Connects back to Week 2's reflection discussion — can models really catch their own mistakes? (<i>Thanks to Rich for sharing this one!</i>)

Next Week

Week 4: HITL Systems

Adding human review into the pipeline.

When should the AI decide alone? When should it ask for help?